

MedFlow – Full-Stack Study Plan

A step-by-step plan to understand and study the entire MedFlow project: backend, frontend, AI integration, and the Separation of Concerns architecture.

Phase 0: Prerequisites (If Needed)

Topic	What to know	Quick check
Python	Async/await, type hints, Pydantic	Read <code>backend/app/models/patient.py</code> and understand the schemas
React + TypeScript	Components, hooks, types	Read <code>frontend/src/App.tsx</code> and one page (e.g. <code>Dashboard.tsx</code>)
REST APIs	GET/POST/PUT, JSON, status codes	Open <code>http://localhost:8000/api/v1/docs</code> when backend is running
Docker basics	<code>docker compose up</code> , services, ports	Run <code>docker compose up -d</code> and <code>docker compose ps</code>

Phase 1: Run the System (Day 1)

Goal: Get everything running and do one full f bw.

1. Setup

- From project root: `./setup.sh` (or `docker compose up -d` then `docker compose exec backend python scripts/init_db.py`).
- Open: **Frontend** <http://localhost:5173>, **API docs** <http://localhost:8000/api/v1/docs>.

2. Login

- Use `nurse1` / `nurse123` (or `doctor1` / `doctor123`, `admin` / `admin123`).

3. One full f bw (Nurse → Doctor)

- **Nurse:** Patients → search/create patient → create session (complaint + state) → submit session.
- **Backend:** Session goes to `submitted` → Celery runs VLM task (if `MEDGEMMA_REMOTE_URL` set) → session becomes `awaiting_doctor` or `vlm_failed`.

- **Doctor:** Doctor Queue → open session → review tabs (History, Session, Files, AI & Chat) → fill diagnosis → close session.

4. Check

- `docker compose logs -f backend` and `docker compose logs -f celery-worker` while you submit a session.

Files to skim: `README.md` (Quick Start + System Workflow), `docker-compose.yml`.

Phase 2: Backend – Core (Days 2–3)

Goal: Understand config, database, auth, and how requests enter the API.

2.1 Config and app entry

Order	Path	Focus
1	<code>backend/app/core/config.py</code>	Env vars: MongoDB, Redis, JWT, <code>MEDGEMMA_REMOTE_URL</code>
2	<code>backend/app/main.py</code>	FastAPI app, CORS, routers, <code>/health</code>
3	<code>backend/app/core/database.py</code>	Motor client, DB name, indexes, <code>get_next_sequence</code>

2.2 Auth and security

Order	Path	Focus
4	<code>backend/app/core/security.py</code>	Password hash/verify, JWT create/decode, <code>get_current_user</code> , <code>require_role</code>
5	<code>backend/app/models/user.py</code>	User, UserCreate, Token, LoginRequest, roles
6	<code>backend/app/api/v1/auth.py</code>	Login, register, <code>/me</code> , logout; how JWT is created and used

Exercise: In API docs, call `POST /api/v1/auth/login`, then `GET /api/v1/auth/me` with the token.

2.3 Data models (Pydantic)

Order	Path	Focus
-------	------	-------

7	<code>backend/app/models/patient.py</code>	Patient, PatientCreate, PatientUpdate, enums, nested (Medication, SurgicalProcedure)
8	<code>backend/app/models/session.py</code>	Session, SessionStatus, SessionCreate, VLM fields, Diagnosis, PendingTests
9	<code>backend/app/models/log.py</code>	LogEntry, LogLevel, LogCategory

Exercise: Map one API request (e.g. create patient) from route → model → service → DB.

Phase 3: Backend – Services and API (Days 4–5)

Goal: Trace one resource end-to-end (e.g. patients or sessions) and see how services and DB interact.

3.1 Patients

Order	Path	Focus
1	<code>backend/app/services/patient_service.py</code>	create_patient, get_patient, update_patient, search_patients, get_patient_portfolio
2	<code>backend/app/api/v1/patients.py</code>	Routes; role checks (<code>require_role</code>); portfolio and nurse VLM masking

Exercise: Create a patient via API docs and via UI; then GET portfolio and see sessions list.

3.2 Sessions

Order	Path	Focus
3	<code>backend/app/services/session_service.py</code>	create_session, update_session, upload_session_file, submit_session (and Celery trigger)
4	<code>backend/app/api/v1/sessions.py</code>	Create, update, file upload, submit, delete; role rules for

4	<code>backend/app/api/v1/sessions.py</code>	submit, delete; role rules for nurse/admin
5	<code>backend/app/services/storage_service.py</code>	GCS vs mock; upload, signed URL, delete

Exercise: Create a session, add a file, submit; watch Celery logs for the VLM task.

3.3 Doctor and dashboard

Order	Path	Focus
6	<code>backend/app/api/v1/doctor.py</code>	Queue (awaiting_doctor, vlm_failed, doctor_reviewing), get_session_for_review (lock), vlm-chat, reanalyze, diagnosis, pending-tests, close (and follow-up creation)
7	<code>backend/app/api/v1/dashboard.py</code>	Role-based stats (nurse / doctor / admin)
8	<code>backend/app/api/v1/admin.py</code>	Logs, log stats, system status (MongoDB + VLM URL check)

Exercise: As doctor, open a session from queue, send one VLM chat message, submit diagnosis, close.

3.4 Logging

Order	Path	Focus
9	<code>backend/app/services/log_service.py</code>	create_log, get_logs, get_logs_count
10	Where <code>log_service</code> is used	auth.py (login/logout), session_service (create/submit), doctor.py (review/close)

Phase 4: Backend – VLM and Celery (Days 6–7)

Goal: Understand how MedGemma is called and how the async task runs.

4.1 MedGemma service

Order	Path	Focus
-------	------	-------

1	backend/app/services/medgemma_service.py	process_initial_session (pro → POST to Colab /predict_text parse), process_doctor_query , process_reanalysis_with_cont _build_initial_prompt , _parse_initial_response
2	backend/app/core/config.py	MEDGEMMA_REMOTE_URL (Colab/ngrok)

Exercise: Set MEDGEMMA_REMOTE_URL (or leave empty and see vlm_failed); run test_medgemma_connection.py if URL is set.

4.2 Celery task

Order	Path	Focus
3	backend/celery_app.py	Broker/backend (Redis), task module
4	backend/app/tasks/vlm_tasks.py	process_session_vlm : set status to vlm_processing → load session/patient → call medgemma_service.process_initial_session → update session (awaiting_doctor or vlm_failed)

Exercise: Submit a session and follow logs in vlm_tasks and medgemma_service ; check DB for vlm_initial_output and session_status .

4.3 Optional: mock VLM

Order	Path	Focus
5	backend/app/services/vlm_service.py	MockVLMService (not used when MedGemma URL is set); structure of response (findings, observations, etc.)

Phase 5: Frontend – Core (Days 8–9)

Goal: Understand app shell, auth, routing, and one full page f bw.

5.1 Entry and config

Order	Path	Focus
1	frontend/index.html	Root div
2	frontend/src/main.tsx	React root, StrictMode
3	frontend/src/App.tsx	Router, QueryClient, Theme, AuthProvider; routes and ProtectedRoute
4	frontend/src/config/api.ts	API_BASE_URL, API_V1_PREFIX
5	frontend/src/utils/axios.ts	Axios instance, base URL, auth header (Bearer token)

5.2 Auth and layout

Order	Path	Focus
6	frontend/src/context/AuthContext.tsx	login, logout, user state, localStorage (token + user)
7	frontend/src/components/ProtectedRoute.tsx	allowedRoles, redirect to login or /unauthorized
8	frontend/src/components/Layout.tsx	Drawer, menu by role, user menu, Outlet

Exercise: Log in as nurse; see only nurse menu; log in as doctor and see Doctor Queue.

5.3 Types and one f b w

Order	Path	Focus
9	frontend/src/types/user.ts	User, UserRole, LoginRequest, LoginResponse
10	frontend/src/types/patient.ts	Patient, PatientSearchResult, PatientCreateRequest
11	frontend/src/types/session.ts	Session, SessionStatus, SessionCreate, UploadedFile, Doctor

Exercise: Trace “create session” from page → service → API; match types to backend models.

Phase 6: Frontend – Pages and Services (Days 10–12)

Goal: Map each main screen to its API and components.

6.1 Services (API layer)

Order	Path	Focus
1	frontend/src/services/userService.ts	login (if used) or rely on AuthContext
2	frontend/src/services/patientService.ts	search, get, create, update, getPatientPortfolio
3	frontend/src/services/sessionService.ts	create, get, update, upload file, submit, delete
4	frontend/src/services/doctorService.ts	getQueue, getSessionForReview, chatWithVLM, reanalyze, submitDiagnosis, setPendingTests, closeSession
5	frontend/src/services/dashboardService.ts	getStats
6	frontend/src/services/adminService.ts	logs, log stats, system status

6.2 Pages

Order	Path	Focus
7	frontend/src/pages/Login.tsx	Form, AuthContext.login, redirect on success
8	frontend/src/pages/Dashboard.tsx	useQuery dashboard stats; role-based stat cards
9	frontend/src/pages/patients/PatientSearch.tsx	Search, create link, table, navigate to profile
10	frontend/src/pages/patients/PatientForm.tsx	Create/edit; validation; submit

10	frontend/src/pages/patients/PatientForm.tsx	submit → patientService
11	frontend/src/pages/patients/PatientProfile.tsx	Portfolio (patient + sessions), create session link
12	frontend/src/pages/sessions/SessionCreate.tsx	Stepper (patient → complaint/state → files); sessionService create/upload/submit
13	frontend/src/pages/sessions/SessionDetail.tsx	View session (nurse); different from doctor review
14	frontend/src/pages/doctor/DoctorQueue.tsx	Queue list, filter “assigned to me”, open SessionReview
15	frontend/src/pages/doctor/SessionReview.tsx	Tabs (History, Session, Files, AI & Chat); VLMChat, DiagnosisForm, close
16	frontend/src/pages/admin/UserManagement.tsx	(If present) user list / create
17	frontend/src/pages/admin/SystemLogs.tsx	Log list, filters, system status

6.3 Shared components

Order	Path	Focus
18	frontend/src/components/FileUpload.tsx	File list, type selector, upload/delete for session
19	frontend/src/components/doctor/VLMChat.tsx	Chat UI, doctorService.chatWith formatVLMResponse
20	frontend/src/components/doctor/DiagnosisForm.tsx	Diagnosis fields, medications, pending submit
21	frontend/src/utils/vlmFormatter.tsx	formatVLMResponse (display VLM output)

Exercise: For each role (nurse, doctor, admin), do one main action and trace it from button → service → API in DevTools Network.

Phase 7: DB and Scripts (Day 13)

Goal: Know the stored shape and how the DB is bootstrapped.

Order	Path	Focus
1	<code>backend/app/core/database.py</code>	Indexes: patients (patient_id, national_id), sessions (session_id, patient_id, status), users (user_id, username)
2	<code>backend/scripts/init_db.py</code>	Counters (user_id, patient_id, session_id); default users (admin, doctor1, doctor2, nurse1)
3	README.md "Database Schema"	patients and sessions collection shape

Exercise: Inspect one patient and one session in MongoDB (e.g. via Compass or `docker compose exec mongodb mongosh`).

Phase 8: Separation of Concerns (Days 14–15)

Goal: Understand the "correct" architecture and why the main app uses a simpler one.

Order	Path	Focus
1	<code>other/Separation of Concerns/README.md</code>	Problem (VLM as classifier), 3 parts (representation, classifier, explanation), what wasn't applied and why
2	<code>other/Separation of Concerns/MedGemma_CXR_-_Proper_Multi_Task_Notebook.ipynb</code>	High-level: how image → MedGemma → embeddings; separate classifier; explanation after decision; why it's complex (chat template, pooling, VRAM)
3	Main app again	MedFlow uses "VLM text directly" as a constrained prototype; separation of

concerns = recommended
production design

Exercise: Write 2–3 sentences (e.g. for thesis) contrasting “what MedFlow does” vs “what the Separation of Concerns design does and why it’s preferred when resources allow.”

Phase 9: Optional Extras

Topic	Path / Note
SISR (super-resolution)	<code>sizr/README.md</code> , <code>sizr/app.py</code> – Wavelet-HAT CXR 4× upscale; not wired into main backend
MedGemma Colab	<code>MedGemma_Colab_Service.ipynb</code> – what runs on Colab and exposes <code>/predict_text</code> for the backend
Pipeline diagram	<code>other/MedGemma Service Pipeline Sample/</code> – sample architecture diagram
Federated / watermarking	<code>other/Federated Learning/</code> , <code>other/Watermarking/</code> – concept docs

Suggested Timeline (Rough)

Phase	Content	Days
0	Prerequisites (as needed)	0–1
1	Run system, one full f bw	1
2	Backend core (conf g, DB, auth, models)	2
3	Backend services and API (patients, sessions, doctor, admin)	2
4	VLM and Celery	2
5	Frontend core (app, auth, layout, types)	2
6	Frontend pages and services	3
7	DB and scripts	1
8	Separation of Concerns	2
9	Optional (SISR, Colab, other/)	as needed

Total: about **2–3 weeks** if you do a bit every day; adjust by skipping or deepening sections.

Quick Reference – Where is X?

I want to...	Look at
Change API base URL	<code>frontend/src/config/api.ts</code> , <code>frontend/.env</code> (<code>VITE_API_BASE_URL</code>)
Add a new API route	<code>backend/app/main.py</code> (<code>include_router</code>), new or existing file under <code>backend/app/api/v1/</code>
Change session status f bw	<code>backend/app/models/session.py</code> (<code>SessionStatus</code>), <code>session_service.py</code> , <code>vlm_tasks.py</code>
Change what nurse/doctor can see	<code>backend/app/api/v1/patients.py</code> (<code>portfolio</code>), <code>sessions.py</code> (<code>get_session</code>), role checks in <code>doctor.py</code>
Add a frontend page	New file under <code>frontend/src/pages/</code> , add route in <code>App.tsx</code> , optional menu in <code>Layout.tsx</code>
See how VLM is called	<code>medgemma_service.py</code> , <code>doctor.py</code> (<code>vlm-chat</code> , <code>reanalyze</code>), <code>vlm_tasks.py</code>
Understand “correct” VLM design	<code>other/Separation of Concerns/README.md</code> and notebook

Good luck with the study